

COP 290

Instructor: Abhilash Jindal

C Lab: Spreadsheet Program

Group Members:

Aaditya Sharma (2023CS10420), Yash Rawat (2023CS50334), Ishan Rehal (2023CS10019)

Semester 2024-2025, Sem II

Abstract

This document provides a comprehensive overview of the design, implementation, and testing of a memory-optimized spreadsheet program in C. The program supports cell assignment, formula evaluation, and efficient recalculation through DFS-based cycle detection and topological sorting. We describe our design decisions—including memory optimizations and refined formula representation—and discuss edge cases and error scenarios covered by our test suite.

1. Introduction

The objective of this project is to develop a command-line spreadsheet program that supports up to 999 rows and 18,278 columns. Each cell stores either an integer or a formula (e.g., `A1=3` or `B1=A1+1`). The interface supports formula entry, scrolling, and additional commands (`disable_output`, `enable_output`, `scroll_to <CELL>`). Automatic recalculation is performed efficiently by updating only the affected cells.

2. System Design and Data Structures

2.1. Cell Representation

Each cell is represented by a structure containing:

- **Cell Value:** An integer storing the cell's value.
- **Visited/Error Flag:** A single `char` encoding both DFS visited state and error status using:
 - `'0'`: Not visited, no error.
 - `'1'`: Not visited, error flag set.
 - `'2'`: Visited, no error.
 - `'3'`: Visited, error flag set.
- **Dependency List:** Instead of storing full pointers, we maintain a list of pairs (each pair consists of two `short ints` representing row and column) for dependent cells. This compact representation minimizes memory overhead.
- **Cell Formula:** A pointer to a structure that represents the current formula. This structure distinguishes among:
 - Constant values,
 - Single cell references,
 - Arithmetic expressions, and
 - Function calls (e.g., `SUM`, `MIN`).

Type codes (e.g., `'1'`, `'5'`, `'8'`) identify the formula type.

2.2. Optimizations in Data Storage

Key optimizations include:

- **Short int for Indices:** Row and column indices are stored as `short int` (2 bytes) instead of `int`.
- **Combined Flag Encoding:** The visited and error flags are merged into one `char` (1 byte).
- **Compact Dependency Storage:** Dependencies are stored as pairs of `short ints` (4 bytes per dependency, 2 for col and 2 for row) instead of full pointers.

3. Algorithmic Strategies

3.1. Cycle Detection via DFS

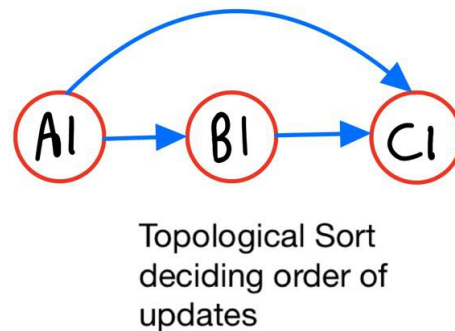
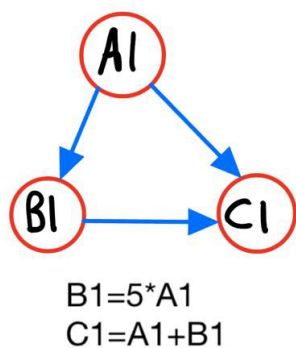
When a new formula is entered (e.g., $A1 = \text{SUM}(B1:C20)$), our DFS-based cycle detection works as follows:

1. **Initiation:** DFS is started at the target cell (A1).
2. **Dependency Check:**
 - For non-range formulas (e.g., $A1 = B1+C1$), DFS checks whether any of the dependency cells (B1 or C1) appear in A1's dependency graph.
 - For range formulas, DFS traverses the dependency graph and checks if any visited cell's coordinates fall within the specified range (e.g., from B1 to C20).
3. **Cycle Reporting:** If a matching dependency is found, a cycle is detected and the update is aborted.
4. **Unmarking:** After DFS, all visited nodes are unmarked (the visited portion of the flag is reset) so that the DFS state is clean for future operations.

3.2. Topological Sorting for Recalculation

Topological sorting ensures that each cell is recalculated only after its prerequisites are updated:

- A DFS-based topological sort is performed to produce a sorted order of cells.
- For example, if B1 depends on A1 and C1 depends on both A1 and B1, then an update to A1 will trigger B1 to update first, followed by C1.
- As nodes are popped from the DFS stack, their visited flags are cleared, ensuring readiness for future DFS calls.



3.3. ERR Propagation

ERR propagation is integrated within the recalculation algorithm:

- An error (ERR) is usually triggered by an invalid operation such as division by zero. When this occurs, the cell is flagged.

- During topological sorting and recalculation, if a dependency cell is flagged with ERR, then its dependents are also marked with ERR.
- Conversely, if an operation successfully computes a value, the error flag is cleared.

3.4. Formula Parsing and Representation

Our parsing module uses regular expressions to handle formulas with any number of spaces (e.g., `A1 = B1 + C1`). The improvements in formula representation include:

- **Type Codes:** The `CellFormula` structure includes explicit type codes to differentiate:
 - Constant values (e.g., `A1=23`),
 - Single cell references (e.g., `A1=B1`),
 - Arithmetic expressions (e.g., `A1=B1+C1`), and
 - Function calls (e.g., `A1=SUM(B1:C20)`).
- **Structured Storage:**
 - Constants are stored as a single value.
 - Cell references are stored as pairs of coordinates.
 - Arithmetic expressions store two pairs (for left and right operands).
 - Function calls store the top-left and bottom-right coordinates of the range.

This clear structure simplifies dependency extraction and update management.

4. Dependency Management and Update Logic

When a cell's formula is updated:

1. **Cycle Detection:** DFS is run from the target cell to check whether any new dependency (or any cell within the new dependency range) is already present in the dependency graph. If a cycle is detected, the update is aborted.
2. **Removing Old Dependencies:** Based on the previous formula, the target cell is removed from the dependency lists of cells it formerly referenced. For example, if the old formula was `A1 = E1+F1`, then A1 is removed from both E1's and F1's dependency lists. An edge case was found in formulas like `A1 = A2+A2` where A1 appears twice in A2's list; in such cases, all duplicate entries of A1 are removed.
3. **Adding New Dependencies:** New dependencies are added by updating the dependency lists (storing pairs of `short ints`) of the cells referenced by the new formula.
4. **Topological Update:** Finally, a DFS-based topological sort is performed to update cells in the correct order. This guarantees that a cell is recalculated only after all its dependencies have been updated.

5. Edge Cases and Error Handling

Our test suite covers several scenarios:

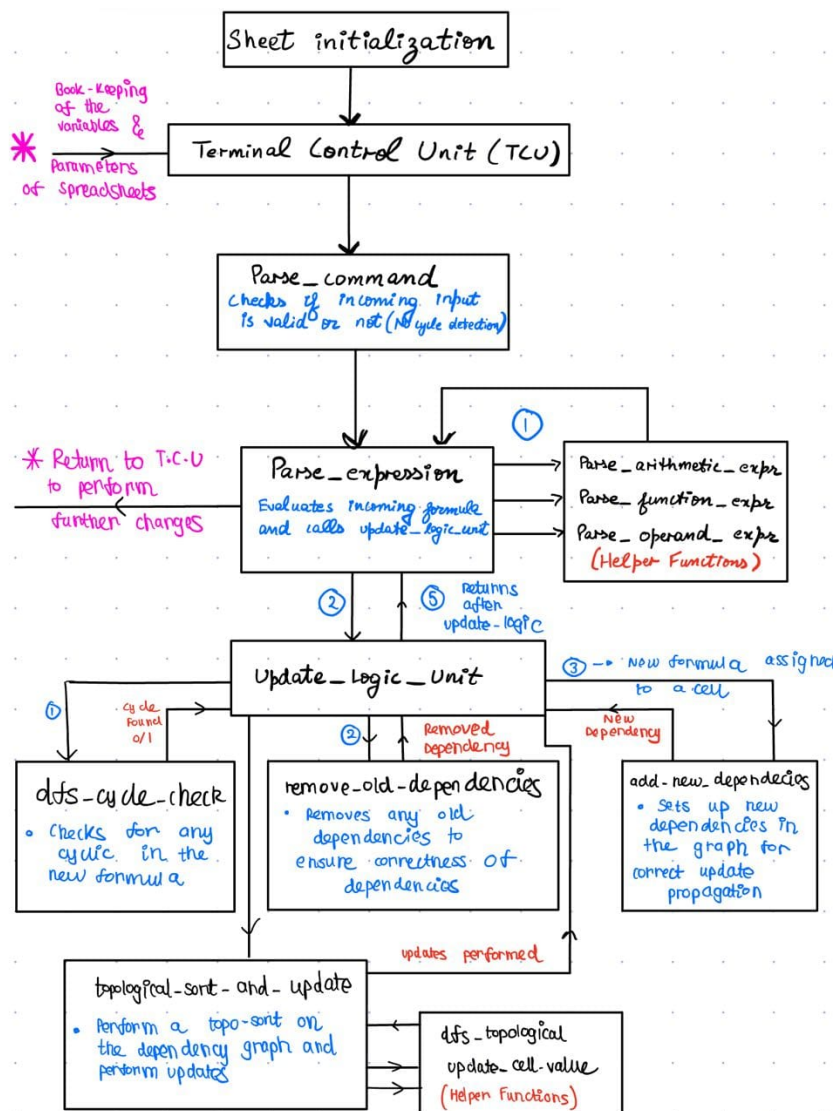
- **Invalid Inputs:** Formulas with incorrect syntax or invalid cell names (e.g., `A0=3`, `a1=2`) are rejected by our regex-based parser.
- **Out-of-Range Cells:** References to cells outside the grid are flagged as errors.
- **Division by Zero:** Cells that encounter division by zero are flagged with ERR. During recalculation, if a dependency is flagged, the error propagates to dependent cells.
- **Circular Dependencies:** Direct or indirect cycles (e.g., `A1 = SUM(B1:C20)` causing a loop) are detected via DFS and the update is aborted.
- **Duplicate Dependencies:** For formulas like `A1 = A2+A2`, duplicate entries are removed from the dependency list to avoid redundant processing.
- **Recalculation Order:** Topological sorting ensures that updates occur in the proper order.

6. Conclusion

Our spreadsheet program is designed to be both robust and memory efficient. Key contributions include:

- **Memory Optimization:** Using `short int` for indices, a single `char` for combined visited/error flags, and compact dependency storage as pairs of `short ints`.
- **Enhanced Formula Representation:** Clear type differentiation for constants, cell references, arithmetic expressions, and function calls that simplifies dependency management.
- **Efficient Algorithms:** DFS-based cycle detection and topological sorting guarantee that cells are updated in the correct order. ERR propagation is integrated so that errors (e.g., division by zero) are flagged and propagated through the dependency chain.
- **Robust Parsing:** Our parser, based on regular expressions, accommodates any number of spaces, ensuring flexibility in user input.

This design not only meets the project requirements but also provides a scalable and efficient foundation for future enhancements.



Video of our Project at [This link](#).

Github Link for our Project is [This link](#).